WEEK 6

PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

1. INTRODUCTION

The Product Review and Rating Management System is designed to collect and manage customer feedback about products purchased from an e-commerce company. It allows customers to provide ratings and comments based on their experiences with different products.

This system stores customer reviews, maintains product ratings, and helps analyze product performance. It also enables the identification of highly rated products, products receiving the most reviews, and products that require improvement.

The Review module is connected to the existing Customer and Product modules using foreign key constraints.

2. OBJECTIVES

The main objectives of this project are:

- To create a Review table for storing customer feedback.
- To maintain product ratings on a scale of 1 to 5.
- To insert, update, and delete customer reviews.
- To retrieve product reviews and customer feedback.
- To calculate average ratings and count reviews for each product.
- To identify highly rated products and products requiring improvement.
- To generate product rating and customer feedback reports.

3. CREATION OF REVIEW TABLE

A Review table was created to store customer reviews and ratings. The table contains REVIEW_ID, CUSTOMER_ID, PRODUCT_ID, RATING, REVIEW_COMMENT, and REVIEW_DATE.

The REVIEW_ID column acts as the primary key. The CUSTOMER_ID and PRODUCT_ID columns are connected to the existing Customer and Product tables through foreign key constraints. A CHECK constraint ensures that ratings remain between 1 and 5, and the rating column does not allow NULL values. The review date is automatically assigned using SYSDATE.

```
SQL> CREATE TABLE REVIEW (REVIEW_ID NUMBER PRIMARY KEY, CUSTOMER_ID VARC
HAR2(20), PRODUCT_ID VARCHAR2(7), RATING NUMBER CONSTRAINT CHK_REVIEW_RA
TING CHECK (RATING BETWEEN 1 AND 5) NOT NULL, REVIEW_COMMENT VARCHAR2(20
0), REVIEW_DATE DATE DEFAULT SYSDATE, CONSTRAINT FK_REVIEW_CUSTOMER FORE
IGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID), CONSTRAINT FK_RE
VIEW_PRODUCT FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT(PRODUCT_ID));

Table created.
```

4. STORAGE OF CUSTOMER FEEDBACK AND RATINGS

4.1 Inserting Customer Reviews

Customer reviews were inserted into the Review table using INSERT statements. Each record contains a unique review ID, customer ID, product ID, rating, review comment, and review date.

Sample feedback includes “Excellent product,” “Good quality,” “Average product,” “Very useful,” and “Poor quality.” Different ratings were assigned to demonstrate customer satisfaction levels.

```
SQL> INSERT INTO REVIEW VALUES (1,'C-2001','P2001',5,'Excellent product',SYSDATE);

1 row created.

SQL> INSERT INTO REVIEW VALUES (2,'C-2002','P2002',4,'Good quality',SYSDATE);

1 row created.

SQL> INSERT INTO REVIEW VALUES (3,'C-2003','P2003',3,'Average product',SYSDATE);

1 row created.

SQL> INSERT INTO REVIEW VALUES (4,'C-2004','P2004',5,'Very useful',SYSDATE);

1 row created.

SQL> INSERT INTO REVIEW VALUES (5,'C-2005','P2005',2,'Poor quality',SYSDATE);

1 row created.

SQL> INSERT INTO REVIEW VALUES (6,'C-2001','P2006',4,'Nice product',SYSDATE);

1 row created.

SQL> INSERT INTO REVIEW VALUES (7,'C-2002','P2007',5,'Amazing quality',SYSDATE);

1 row created.
```

4.2 Displaying All Reviews

The SELECT statement was used to retrieve all records from the Review table. The output displays the stored customer IDs, product IDs, ratings, review comments, and review dates.

This verifies that the customer feedback has been stored in the table.

```
SQL> SELECT * FROM REVIEW;

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------
         1 C-2001               P2001            5
Excellent product
21-OCT-26

         2 C-2002               P2002            4
Good quality
21-OCT-26

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------

         3 C-2003               P2003            3
Average product
21-OCT-26

         4 C-2004               P2004            5
Very useful
```

Additional review data was inserted, including a review for product P2010. The COMMIT statement was executed to save the changes permanently.

```
SQL> INSERT INTO REVIEW VALUES (10,'C-2005','P2010',4,'Worth buying',SYSDATE);

1 row created.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM REVIEW;

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------
         1 C-2001               P2001            5
Excellent product
21-OCT-26

         2 C-2002               P2002            4
Good quality
21-OCT-26

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------

         3 C-2003               P2003            3
Average product
```

4.3 Updating Review Comments

The UPDATE statement was used to modify an existing review comment. The comment associated with REVIEW_ID 1 was changed to “Excellent quality and performance.”

The updated record was retrieved using a SELECT statement to verify that the modification was successful.

```
SQL> UPDATE REVIEW SET REVIEW_COMMENT='Excellent quality and performance' WHERE REVIEW_ID=1;

1 row updated.

SQL> SELECT * FROM REVIEW WHERE REVIEW_ID=1;

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------
         1 C-2001               P2001            5
Excellent quality and performance
21-OCT-26
```

4.

Th
confirmed that one row was deleted.

This demonstrates how inappropriate or invalid reviews can be removed from the database.

```
SQL> DELETE FROM REVIEW WHERE REVIEW_ID=9;

1 row deleted.
```

5. RETRIEVAL OF PRODUCT REVIEW DETAILS

5.1 Displaying All Reviews for a Product

A SELECT query was used to retrieve reviews for product P2001. The result displayed the product ID, customer ID, rating, review comment, and review date for the matching record.

This helps retrieve feedback for an individual product.

```
SQL> SELECT * FROM REVIEW WHERE PRODUCT_ID='P2001';

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
--------
REVIEW_DA
---------
         1 C-2001               P2001            5
Excellent quality and performance
21-OCT-26
```

5.2 Displaying Customer Names with Their Reviews

A JOIN operation was performed between the Customer and Review tables using CUSTOMER_ID. The query retrieves customer first names, last names, product IDs, ratings, and review comments.

The output connects customer information with the feedback provided by each customer. This makes it easier to understand which customer reviewed a particular product.

```
SQL> SELECT C.FIRST_NAME, C.LAST_NAME, R.PRODUCT_ID, R.RATING, R.REVIEW_COMMENT FROM CUSTOMER C JOIN REVIEW R ON C.CUSTOMER_ID=R.CUSTOMER_ID;

FIRST_NAME           LAST_NAME            PRODUCT     RATING
-------------------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
Lira                 James                P2001            5
Excellent quality and performance

Lira                 James                P2006            4
Nice product

Kenny                Robert               P2002            4
Good quality


FIRST_NAME           LAST_NAME            PRODUCT     RATING
-------------------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
Kenny                Robert               P2007            5
Amazing quality

Kairo                William              P2003            3
Average product

Kairo                William              P2008            3
Okay product


FIRST_NAME           LAST_NAME            PRODUCT     RATING
-------------------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
Suxi                 Martin               P2004            5
Very useful

Blissy               Thomas               P2005            2
Poor quality

Blissy               Thomas               P2010            4
Worth buying

9 rows selected.
```

5.3 Finding Products Having the Maximum Number of Reviews

An aggregate query using COUNT and MAX was executed to identify products having the maximum number of reviews.

The output showed that each of the nine remaining products had one review. Therefore, all the listed products shared the maximum review count.

```
SQL> SELECT PRODUCT_ID, COUNT(*) AS TOTAL_REVIEWS FROM REVIEW GROUP BY PRODUCT_ID
 HAVING COUNT(*)=(SELECT MAX(COUNT(*)) FROM REVIEW GROUP BY PRODUCT_ID);

PRODUCT TOTAL_REVIEWS
------- -------------
P2001                 1
P2002                 1
P2003                 1
P2004                 1
P2005                 1
P2006                 1
P2007                 1
P2008                 1
P2010                 1

9 rows selected.
```

5.4 Displaying Recent Customer Feedback

The Review table was sorted by REVIEW_DATE in descending order to display recent customer feedback first.

This query helps retrieve reviews according to their recorded dates. The output displays the review details, including the customer ID, product ID, rating, comment, and review date.

```
SQL> SELECT * FROM REVIEW ORDER BY REVIEW_DATE DESC;

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------
         1 C-2001               P2001            5
Excellent quality and performance
21-OCT-26

         2 C-2002               P2002            4
Good quality
21-OCT-26

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------

         3 C-2003               P2003            3
Average product
21-OCT-26

         4 C-2004               P2004            5
Very useful

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
REVIEW_DA
---------
21-OCT-26

        10 C-2005               P2010            4
Worth buying
21-OCT-26
```

6. CALCULATION OF AVERAGE PRODUCT RATINGS

6.1 Calculating the Average Rating for Each Product

The AVG aggregate function was used with GROUP BY PRODUCT_ID to calculate the average rating for each product.

The output displayed the average rating of every product based on the reviews available in the database. This helps compare the rating performance of different products.

```
SQL> SELECT PRODUCT_ID, AVG(RATING) AS AVERAGE_RATING FROM REVIEW GROUP BY PRODUC
T_ID;

PRODUCT AVERAGE_RATING
------- --------------
P2001                5
P2002                4
P2003                3
P2004                5
P2005                2
P2006                4
P2007                5
P2008                3
P2010                4

9 rows selected.

SQL> SELECT PRODUCT_ID, COUNT(*) AS TOTAL_REVIEWS FROM REVIEW GROUP BY PRODUCT_ID
;

PRODUCT TOTAL_REVIEWS
------- -------------
P2001               1
P2002               1
P2003               1
P2004               1
P2005               1
P2006               1
P2007               1
P2008               1
P2010               1

9 rows selected.
```

6.2 Counting the Number of Reviews per Product

The COUNT aggregate function was used to calculate the total number of reviews for each product. The results were grouped by PRODUCT_ID.

The output showed nine products, each having one review after the deletion of REVIEW_ID 9. This demonstrates how the number of reviews can be analyzed for individual products.

```
SQL> SELECT PRODUCT_ID, RATING FROM REVIEW WHERE RATING=(SELECT MAX(RATING) FROM
REVIEW);

PRODUCT     RATING
------- ----------
P2001            5
P2004            5
P2007            5

SQL> SELECT PRODUCT_ID, AVG(RATING) AS AVERAGE_RATING FROM REVIEW GROUP BY PRODUC
T_ID HAVING AVG(RATING)>4;

PRODUCT AVERAGE_RATING
------- --------------
P2001                5
P2004                5
P2007                5
```

6.3 Finding the Highest-Rated Products

The MAX aggregate function was used with GROUP BY PRODUCT_ID to identify the highest rating received by each product.

The output displayed the highest rating for every product. Products P2001, P2004, and P2007 received the maximum rating of 5.

```
SQL> SELECT PRODUCT_ID, COUNT(*) AS TOTAL_REVIEWS
  2  FROM REVIEW
  3  GROUP BY PRODUCT_ID;

PRODUCT TOTAL_REVIEWS
------- -------------
P2001               1
P2002               1
P2003               1
P2004               1
P2005               1
P2006               1
P2007               1
P2008               1
P2010               1

9 rows selected.

SQL> SELECT PRODUCT_ID, MAX(RATING) AS HIGHEST_RATING
  2  FROM REVIEW
  3  GROUP BY PRODUCT_ID;

PRODUCT HIGHEST_RATING
------- --------------
P2001                5
P2002                4
P2003                3
P2004                5
P2005                2
P2006                4
P2007                5
P2008                3
P2010                4

9 rows selected.
```

6.4 Identifying Products with an Average Rating Above 4

The GROUP BY and HAVING clauses were used to identify products with an average rating greater than 4.

The output displayed products P2001, P2004, and P2007, each with an average rating of 5. These products can be considered highly rated based on the available reviews.

```
SQL> SELECT PRODUCT_ID, AVG(RATING) AS AVERAGE_RATING
  2  FROM REVIEW
  3  GROUP BY PRODUCT_ID
  4  HAVING AVG(RATING) >= 4
  5  ORDER BY AVERAGE_RATING DESC;

PRODUCT AVERAGE_RATING
------- --------------
P2001                5
P2004                5
P2007                5
P2006                4
P2010                4
P2002                4

6 rows selected.
```

7. IDENTIFICATION OF HIGHLY RATED PRODUCTS

Product ratings can be used to evaluate customer satisfaction and product performance. Products receiving consistently high ratings may indicate positive customer experiences, while lower ratings may highlight areas that require attention.

Based on the available review data, products P2001, P2004, and P2007 received ratings of 5. These products were identified as the highest-rated products in the current dataset.

Products with lower ratings can be examined further to understand customer concerns and identify possible improvements.

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS
  2  FROM REVIEW
  3  WHERE RATING = 5;

FIVE_STAR_RATINGS
-----------------
                3

SQL> SELECT COUNT(*) AS FOUR_STAR_RATINGS
  2  FROM REVIEW
  3  WHERE RATING = 4;

FOUR_STAR_RATINGS
-----------------
                3

SQL> SELECT COUNT(*) AS LOW_RATED_REVIEWS
  2  FROM REVIEW
  3  WHERE RATING <= 2;

LOW_RATED_REVIEWS
-----------------
                1

SQL> SELECT RATING, COUNT(*) AS TOTAL_REVIEWS
  2  FROM REVIEW
  3  GROUP BY RATING
  4  ORDER BY RATING DESC;

    RATING TOTAL_REVIEWS
---------- --------------
         5              3
         4              3
         3              2
```

8. PRODUCT RATING ANALYSIS REPORT

The Product Rating Analysis report focuses on product performance using the number of reviews and average ratings.

The report can include the following details:

- Product name or product ID.
- Total number of reviews.
- Average product rating.

The aggregate queries performed in this project provide the review counts and rating information needed for this analysis.

```
SQL> SELECT P.PRODUCT_NAME,
  2         COUNT(R.REVIEW_ID) AS TOTAL_REVIEWS,
  3         AVG(R.RATING) AS AVERAGE_RATING
  4  FROM PRODUCT P
  5  JOIN REVIEW R ON P.PRODUCT_ID = R.PRODUCT_ID
  6  GROUP BY P.PRODUCT_NAME;

PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
Brake Pad Set
            1              5

Castrol Engine Oil
            1              4

Car Phone Holder
            1              3


PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
Bike Crash Guard
            1              5

Steel Helmet
            1              2

Exide Car Battery
            1              4


PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
Leather Seat Cover
            1              5

Premium Floor Mat
            1              3

New Car Perfume
            1              4


9 rows selected.
```

9. CUSTOMER FEEDBACK ANALYSIS REPORT

The Customer Feedback Analysis report helps identify the products that receive the most reviews, the highly rated products, and the products that may require improvement.

The results show that the nine remaining products have one review each. Products P2001, P2004, and P2007 achieved ratings of 5, while products with lower ratings may need further examination.

This analysis can help the company understand customer opinions and make decisions about product quality and customer satisfaction.

```
SQL> SELECT PRODUCT_ID, AVG(RATING) AS AVERAGE_RATING
  2  FROM REVIEW
  3  GROUP BY PRODUCT_ID
  4  HAVING AVG(RATING) >= 4
  5  ORDER BY AVERAGE_RATING DESC;

PRODUCT AVERAGE_RATING
------- --------------
P2001                5
P2004                5
P2007                5
P2006                4
P2010                4
P2002                4

6 rows selected.
```

```
SQL> SELECT PRODUCT_ID, AVG(RATING) AS AVERAGE_RATING
  2  FROM REVIEW
  3  GROUP BY PRODUCT_ID
  4  HAVING AVG(RATING) < 3
  5  ORDER BY AVERAGE_RATING ASC;

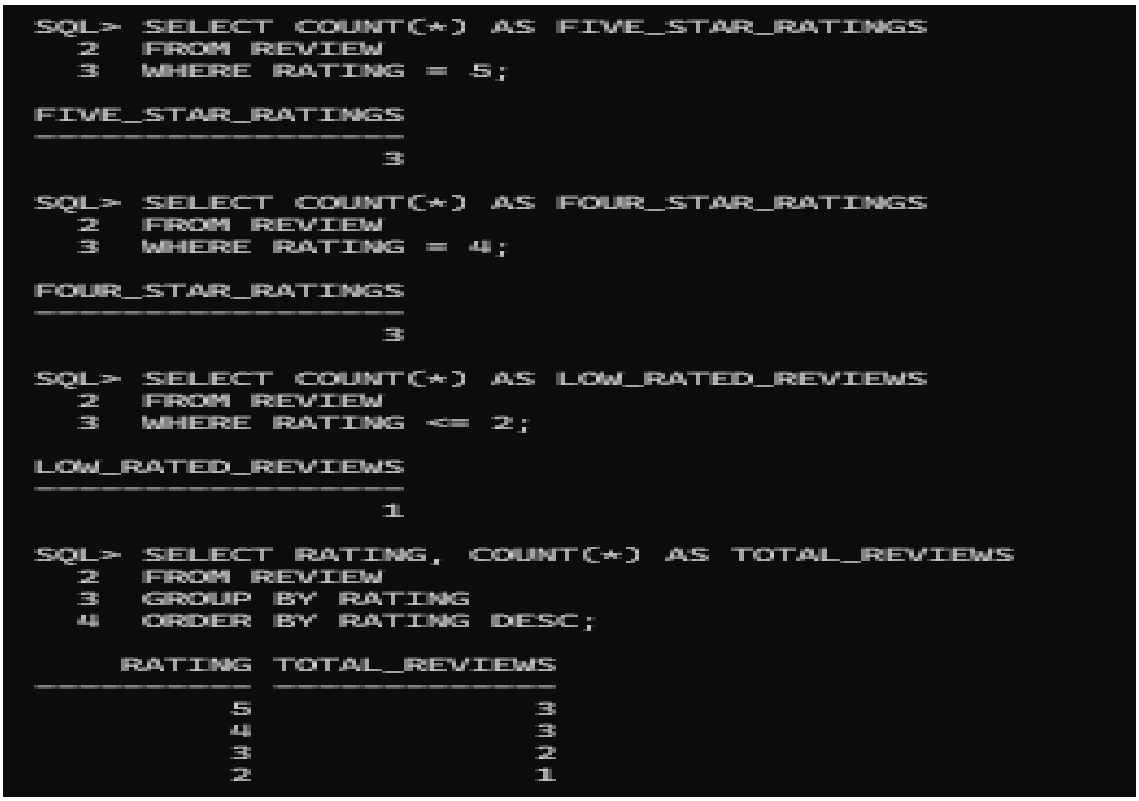
PRODUCT AVERAGE_RATING
------- --------------
P2005                2
```

10. RATING DISTRIBUTION REPORT

The Rating Distribution report analyzes how customer reviews are distributed across different rating levels.

The number of five-star ratings and four-star ratings can be calculated using the COUNT function with suitable conditions. Low-rated reviews can also be counted to identify products or experiences that may require improvement.

The complete rating distribution can be obtained by grouping the reviews according to their rating values. This helps the company understand overall customer satisfaction.



```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS
  2  FROM REVIEW
  3  WHERE RATING = 5;

FIVE_STAR_RATINGS
-----------------
                3

SQL> SELECT COUNT(*) AS FOUR_STAR_RATINGS
  2  FROM REVIEW
  3  WHERE RATING = 4;

FOUR_STAR_RATINGS
-----------------
                3

SQL> SELECT COUNT(*) AS LOW_RATED_REVIEWS
  2  FROM REVIEW
  3  WHERE RATING <= 2;

LOW_RATED_REVIEWS
-----------------
                1

SQL> SELECT RATING, COUNT(*) AS TOTAL_REVIEWS
  2  FROM REVIEW
  3  GROUP BY RATING
  4  ORDER BY RATING DESC;

    RATING TOTAL_REVIEWS
---------- -------------
         5             3
         4             3
         3             2
         2             1
```

11. RESULT

The Product Review and Rating Management System was implemented successfully using SQL in Oracle. The Review table was created with primary key, foreign key, and CHECK constraints. Customer reviews were inserted, updated, retrieved, and deleted successfully.

Aggregate functions such as COUNT, AVG, and MAX were used to analyze product reviews and ratings. JOIN operations were used to retrieve customer names with their reviews, while GROUP BY and HAVING clauses helped identify highly rated products.

The project demonstrates how a database system can store customer feedback, analyze product performance, and support better decision-making in an e-commerce environment.